전북지역혁신플랫폼
Jeonbuk Regional Innovation Platform

JST

지능형모빌리티 설계

1주차 CMake



* 본 영상은 무단 배포할 수 없습니다.

학습내용

1. CMake 사용하기
2. CrossPlatform 컴파일

학습목표

- 컴파일과 빌드에 대해 이해하고 설명할 수 있다.
- CMake를 이용해 실행 파일을 생성할 수 있다.

CMake 사용하기

CrossPlatform 컴파일

Cmake란?

정의

- 실행 파일을 생성해주는 크로스 컴파일러 프로그램
- 운영체제에 관계없이 make 파일을 생성
- 프로젝트에 CMakeLists.txt 파일에 작성
- 최상위 디렉토리에 하나의 CMakeLists.txt를 작성하거나
  각 디렉토리별로 각각 CMakeLists.txt 파일을 작성해 관리

Cmake란?

CMake 사용 예제

```
$ mkdir ~/cmake_example
$ cd ~/cmake_example
$ code ./CMakeLists.txt
```

Cmake란?

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8) # cmake 최소 필요 버전 정의

project(cmake_example) # 프로젝트 이름

# 실행 파일
add_executable(cmake_example src/cmake_example.cpp)
```

Cmake란?

- cmake_example.cpp

```
$ mkdir src
$ cd src
$ code cmake_example.cpp
```

- CMakeLists.txt에 작성한 경로와 동일해야 함

Cmake란?

cmake_example.cpp

```cpp
#include <iostream>

int main() {
 std::cout << "Hello, this is CMake example!!" << std::endl; // 터미널에 출력
 return 0;
}
```

Build

특징

- 프로젝트 최상위 디렉토리에서 build라는 디렉토리 생성
- build 안에서 cmake를 통해 빌드

```
$ cd ~/cmake_example
$ mkdir build
$ cd build
$ cmake ..
$ make
```

Cmake 실행

실행 방법

build 디렉토리 안의 파일 목록을 보면 실행 파일인 cmake_example 존재

```
jairlab@jairlab:~/cmake_example/build$ ls
CMakeCache.txt  cmake_example  CMakeFiles  cmake_install.cmake  Makefile
jairlab@jairlab:~/cmake_example/build$
```

```
$ ./cmake_example
```

출력

Hello, this is CMake example!!

Cmake 실행

디렉토리 및 라이브러리 추가

- CMakeLists.txt에 디렉토리 및 라이브러리를 추가

Cmake 실행

헤더파일 생성

```
$ cd ~/cmake_example
$ mkdir include
$ cd include
$ code ./add_number.hpp
```

Cmake 실행

add_number.hpp

```
#pragma once // 이 파일이 여러 번 포함되더라도 중복되지 않도록 방지하는 지시문

#include <iostream> // 입출력 기능을 사용하기 위해

double add(double num1, double num2); //함수 선언
```

Cmake 실행

라이브러리 파일 생성

```
$ cd ~/cmake_example/src
$ code ./add_number.cpp
```

Cmake 실행

add_number.cpp

```cpp
#include "add_number.hpp" // 헤더파일

double add(double num1, double num2) // 함수 정의
{
    double result  = num1 + num2;
    return result;
}
```

Cmake 실행

실행 파일 생성

```
$ cd ~/cmake_example/src
$ code ./add_main.cpp
```

Cmake 실행

add_main.cpp

```cpp
#include "add_number.hpp" // 사용할 함수가 선언된 헤더파일

int main(int argc, char *argv[] ) {

    if(argc < 3) {
        std::cout<<"Usage: "<<argv[0]<<" num1 num2 "<<std::endl;  // 실행파일 실행 시 숫자 두개를 받았는지 확인
        return 1;
    }
    double num1 = atof(argv[1]);
    double num2 = atof(argv[2]);
    double result = add(num1, num2); // add_number에 정의된 함수 사용
    std::cout<<"Result: "<<num1<<"+"<<num2<<" = "<<result<<std::endl; // 결과 출력
    return 0;
}
```

Cmake 실행

CMaKeLists.txt

```
cmake_minimum_required(VERSION 3.8) # 빌드에 필요한 최소 cmake 버전

project(cmake_example) # 프로젝트 이름

# 실행 파일
add_executable(cmake_example src/cmake_example.cpp) # main 함수가 있는 코드

include_directories("include") # include 디렉토리를 포함

add_library(AddNumber src/add_number.cpp) # 라이브러리 추가

add_executable(add_num src/add_main.cpp) # 실행 파일

Target_link_libraries(add_num AddNumber) # 실행 파일에 라이브러리 링크
                                         라이브러리 이름
```

Cmake 실행

실행

```
$ ./add_num 1.5 2.5
```

출력
Result: 1.5 + 2.5 = 4



정리하기

리눅스 및 ROS2

리눅스 기본 명령어

- 리눅스는 터미널을 사용함
- 터미널의 기본 명령어

개발을 위한 기본 환경 설정

- 손쉬운 개발 환경을 위한 VSCode와 Terminator



정리하기

CMake

CMake 기본 개념

- C언어로 작성된 코드를 실행 파일로 생성해주는 Cross Compiler
- 다양한 운영체제에서 같은 make 파일을 생성
- CMakeLists라는 파일을 이용하여 관리함